

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

Aim: Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

Cli input

```
import math
import random

# =====
# BACKWARD COMPATIBLE MATH FUNCTIONS
# =====

def extended_gcd(a, b):
    """Extended Euclidean Algorithm to find modular inverse."""
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd, x, y

def modular_inverse(e, phi):
    """Calculates d such that (d * e) % phi == 1. Works on all Python
versions."""
    gcd, x, y = extended_gcd(e, phi)
    if gcd != 1:
        raise ValueError("Modular inverse does not exist")
    else:
        return x % phi

def is_prime(n, k=5):
    """Miller-Rabin primality test."""
    if n < 2: return False
    if n in (2, 3): return True
    if n % 2 == 0: return False

    r, s = 0, n - 1
    while s % 2 == 0:
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
    r += 1
    s //= 2
for _ in range(k):
    a = random.randrange(2, n - 1)
    x = pow(a, s, n)
    if x == 1 or x == n - 1:
        continue
    for _ in range(r - 1):
        x = pow(x, 2, n)
        if x == n - 1:
            break
    else:
        return False
return True

def generate_prime(bits=16):
    """Generates an n-bit prime number."""
    while True:
        p = random.getrandbits(bits)
        p |= (1 << bits - 1) | 1
        if is_prime(p):
            return p

def generate_keypair(bits=16):
    """Generates a public and private keypair safely."""
    p = generate_prime(bits)
    q = generate_prime(bits)

    n = p * q
    phi = (p - 1) * (q - 1)

    e = 65537
    if math.gcd(e, phi) != 1:
        e = 3
        while math.gcd(e, phi) != 1:
            e += 2

    # Using our custom function instead of pow(e, -1, phi)
    d = modular_inverse(e, phi)
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
return ((e, n), (d, n))

def encrypt(public_key, plaintext):
    """Encrypts a string using the public key."""
    e, n = public_key
    return [pow(ord(char), e, n) for char in plaintext]

def decrypt(private_key, ciphertext):
    """Decrypts a list of integers using the private key."""
    d, n = private_key
    plain = [chr(pow(char, d, n)) for char in ciphertext]
    return ''.join(plain)

# =====
# RUNNABLE CLI INTERACTION
# =====

if __name__ == "__main__":
    print("\n--- RSA Encryption/Decryption CLI ---")
    print("Generating RSA keypair...")
    public, private = generate_keypair(bits=16)
    print(f"Public Key (e, n): {public}")
    print(f"Private Key (d, n): {private}")

    while True:
        print("\nOptions:")
        print("1. Encrypt a message")
        print("2. Decrypt a message")
        print("3. Exit")
        choice = input("Enter choice (1/2/3): ").strip()

        if choice == '1':
            message = input("Enter message to encrypt: ")
            encrypted_msg = encrypt(public, message)
            print(f"Encrypted message (Ciphertext integers): {encrypted_msg}")
        elif choice == '2':
            cipher_input = input("Enter comma-separated ciphertext integers: ")
            ciphertext = [int(x) for x in cipher_input.split(',')]
            plain_text = decrypt(private, ciphertext)
            print(f"Decrypted message: {plain_text}")
        elif choice == '3':
            break
```

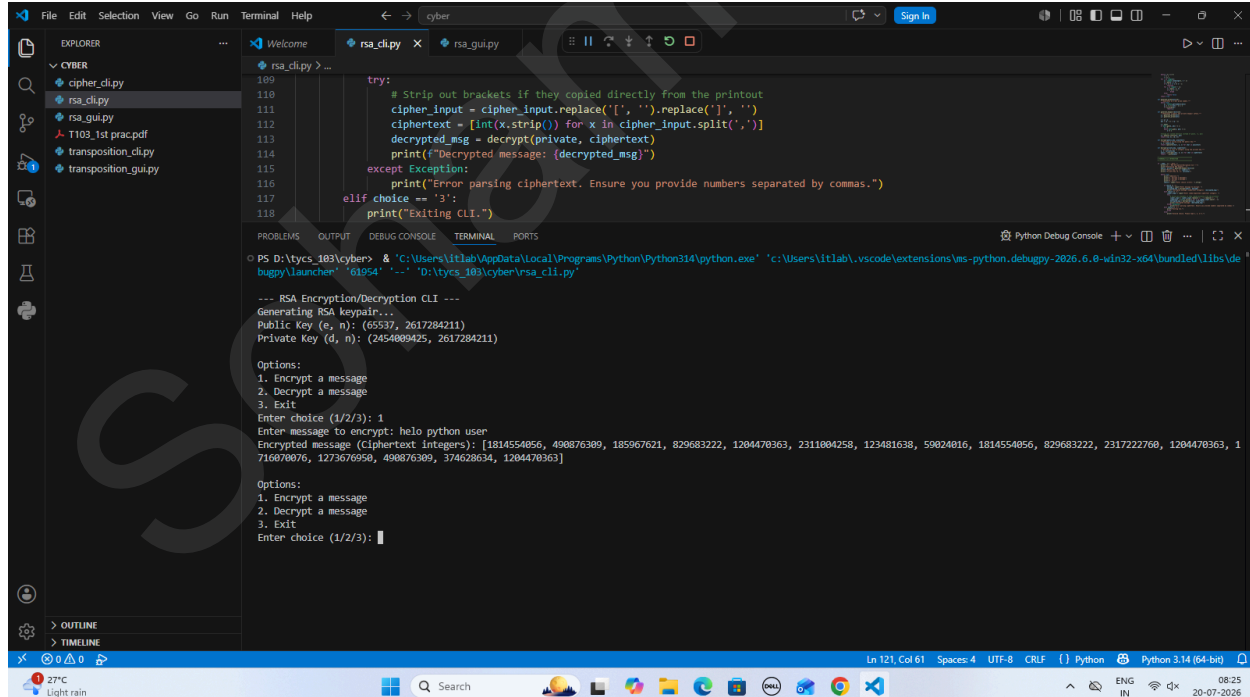
SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
try:
    # Strip out brackets if they copied directly from the
    printout
    cipher_input = cipher_input.replace('[', '').replace(']',
    '')

    ciphertext = [int(x.strip()) for x in
    cipher_input.split(',')
    ]
    decrypted_msg = decrypt(private, ciphertext)
    print(f"Decrypted message: {decrypted_msg}")
except Exception:
    print("Error parsing ciphertext. Ensure you provide
    numbers separated by commas.")
elif choice == '3':
    print("Exiting CLI.")
    break
else:
    print("Invalid choice. Please type 1, 2, or 3.")
```

Output



```
PS D:\tycs_103\cybers > & 'C:\Users\l1lab\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\l1lab\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\de
buggy\launcher' '61954' '--' 'D:\tycs_103\cyber\rsa_cli.py'

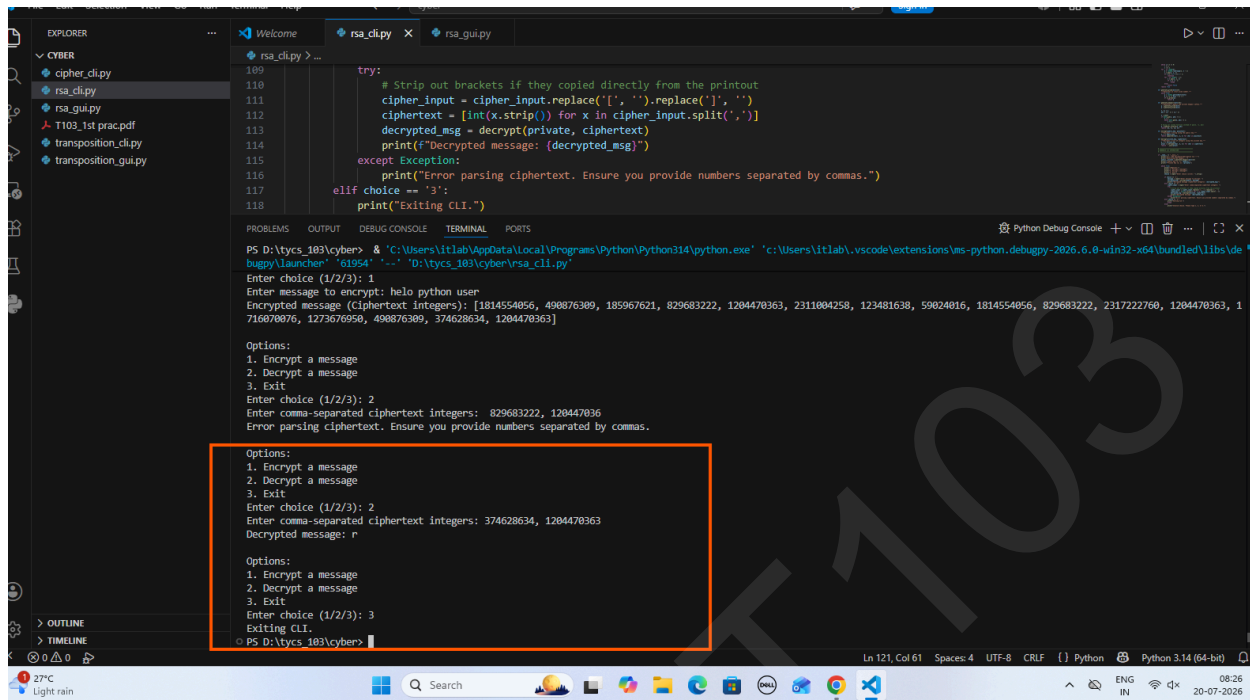
--- RSA Encryption/Decryption CLI ---
Generating RSA keypair...
Public Key (e, n): (65537, 2617284211)
Private Key (d, n): (2454089425, 2617284211)

Options:
1. Encrypt a message
2. Decrypt a message
3. Exit
Enter choice (1/2/3): 1
Enter message to encrypt: hello python user
Encrypted message (Ciphertext integers): [1814554056, 490876380, 185967621, 829683222, 1204470363, 2311004258, 123481638, 59024016, 1814554056, 829683222, 2317222760, 1204470363, 1
716070076, 1273676950, 490876380, 374628634, 1204470363]

Options:
1. Encrypt a message
2. Decrypt a message
3. Exit
Enter choice (1/2/3):
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security



The screenshot shows a VS Code editor with two files: `rsa_cli.py` and `rsa_gui.py`. The `rsa_cli.py` file contains the following code:

```
109     try:
110         # Strip out brackets if they copied directly from the printout
111         cipher_input = cipher_input.replace('[', '').replace(']', '')
112         ciphertext = [int(x.strip()) for x in cipher_input.split(',')]
113         decrypted_msg = decrypt(private, ciphertext)
114         print(f"Decrypted message: {decrypted_msg}")
115     except Exception:
116         print("Error parsing ciphertext. Ensure you provide numbers separated by commas.")
117     elif choice == '3':
118         print("Exiting CLI.")
```

The terminal output shows the execution of the CLI program:

```
PS D:\tycs_103\cyber> & 'C:\Users\itilab\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\itilab\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\11bs\de
bugpy\launcher' '61954' '--' 'D:\tycs_103\cyber\rsa_cli.py'
Enter message to encrypt: halo python user
Encrypted message (ciphertext integers): [1814554056, 490876309, 185967621, 829683222, 1204470363, 2311004258, 123481638, 59024016, 1814554056, 829683222, 2317222760, 1204470363, 1
710070076, 1273676950, 490876309, 374628634, 1204470363]
Options:
1. Encrypt a message
2. Decrypt a message
3. Exit
Enter choice (1/2/3): 2
Enter comma-separated ciphertext integers: 829683222, 120447036
Error parsing ciphertext. Ensure you provide numbers separated by commas.
Options:
1. Encrypt a message
2. Decrypt a message
3. Exit
Enter choice (1/2/3): 2
Enter comma-separated ciphertext integers: 374628634, 1204470363
Decrypted message: r
Options:
1. Encrypt a message
2. Decrypt a message
3. Exit
Enter choice (1/2/3): 3
Exiting CLI.
PS D:\tycs_103\cyber>
```

Gui input

```
import math
import random
import re
import tkinter as tk
from tkinter import messagebox

# =====
# 1. CORE CRYPTOGRAPHIC ENGINE
# =====

def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd, x, y

def modular_inverse(e, phi):
    gcd, x, y = extended_gcd(e, phi)
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
if gcd != 1:
    raise ValueError("Modular inverse does not exist")
return x % phi

def is_prime(n, k=5):
    if n < 2: return False
    if n in (2, 3): return True
    if n % 2 == 0: return False
    r, s = 0, n - 1
    while s % 2 == 0:
        r += 1
        s //= 2
    for _ in range(k):
        a = random.randrange(2, n - 1)
        x = pow(a, s, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True

def generate_prime(bits=16):
    while True:
        p = random.getrandbits(bits)
        p |= (1 << bits - 1) | 1
        if is_prime(p):
            return p

def generate_keypair(bits=16):
    p = generate_prime(bits)
    q = generate_prime(bits)
    n = p * q
    phi = (p - 1) * (q - 1)

    e = 65537
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
if math.gcd(e, phi) != 1:
    e = 3
    while math.gcd(e, phi) != 1:
        e += 2

d = modular_inverse(e, phi)
return ((e, n), (d, n))

def encrypt(public_key, plaintext):
    e, n = public_key
    return [pow(ord(char), e, n) for char in plaintext]

def decrypt(private_key, ciphertext):
    d, n = private_key
    plain = [chr(pow(char, d, n)) for char in ciphertext]
    return ''.join(plain)

# =====
# 2. MODERN FLAT UI DESIGN
# =====

class EnterpriseRSAUI:
    def __init__(self, root):
        self.root = root
        self.root.title("Enterprise RSA Cryptography Suite")
        self.root.geometry("850x750")
        self.root.resizable(False, False)

        # Color Palette (Modern Clean UI)
        self.bg_color = "#F3F4F6"           # Light gray background
        self.card_color = "#FFFFFF"         # White panel background
        self.text_main = "#111827"         # Dark gray/black text
        self.text_muted = "#6B7280"        # Muted gray text
        self.accent_blue = "#2563EB"        # Primary button
        self.accent_blue_hover = "#1D4ED8"
        self.accent_green = "#10B981"      # Decrypt button
        self.accent_green_hover = "#059669"
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
# Fonts
self.font_title = ("Segoe UI", 12, "bold")
self.font_label = ("Segoe UI", 10, "bold")
self.font_text = ("Segoe UI", 11)
self.font_mono = ("Consolas", 11)

self.root.configure(bg=self.bg_color)

self.public_key = None
self.private_key = None

self.build_ui()
self.generate_keys()

def build_ui(self):
    # --- MAIN PADDING CONTAINER ---
    main_pad = tk.Frame(self.root, bg=self.bg_color, padx=30, pady=25)
    main_pad.pack(fill=tk.BOTH, expand=True)

    # --- HEADER ---
    header_frame = tk.Frame(main_pad, bg=self.bg_color)
    header_frame.pack(fill=tk.X, pady=(0, 20))
    tk.Label(header_frame, text="RSA-16 ENCRYPTION ENGINE",
font=("Segoe UI", 16, "bold"), bg=self.bg_color,
fg=self.text_main).pack(side=tk.LEFT)

    btn_regen = tk.Button(header_frame, text="🔄 Regenerate Keys",
font=self.font_label, bg="#E5E7EB", fg=self.text_main,
relief="flat", padx=15, pady=5,
cursor="hand2", command=self.generate_keys)
    btn_regen.pack(side=tk.RIGHT)

    # --- KEY CARDS SECTION ---
    keys_frame = tk.Frame(main_pad, bg=self.bg_color)
    keys_frame.pack(fill=tk.X, pady=(0, 20))

    # Public Key Panel
    pub_panel = tk.Frame(keys_frame, bg=self.card_color, padx=15,
pady=15, highlightbackground="#E5E7EB", highlightthickness=1)
```


SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
pub_panel.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=(0, 10))
    tk.Label(pub_panel, text="PUBLIC KEY (e, n)",
font=self.font_label, bg=self.card_color,
fg=self.accent_blue).pack(anchor=tk.W)
    tk.Label(pub_panel, text="Used to encrypt messages. Share this
publicly.", font=("Segoe UI", 9), bg=self.card_color,
fg=self.text_muted).pack(anchor=tk.W, pady=(0, 10))

    self.pub_entry = tk.Entry(pub_panel, font=self.font_mono,
bg="#F9FAFB", fg=self.text_main, relief="flat")
    self.pub_entry.pack(fill=tk.X, ipady=5)

# Private Key Panel
priv_panel = tk.Frame(keys_frame, bg=self.card_color, padx=15,
pady=15, highlightbackground="#E5E7EB", highlightthickness=1)
    priv_panel.pack(side=tk.RIGHT, fill=tk.X, expand=True, padx=(10,
0))
    tk.Label(priv_panel, text="PRIVATE KEY (d, n)",
font=self.font_label, bg=self.card_color, fg="#DC2626").pack(anchor=tk.W)
    tk.Label(priv_panel, text="Used to decrypt messages. Keep this
secure.", font=("Segoe UI", 9), bg=self.card_color,
fg=self.text_muted).pack(anchor=tk.W, pady=(0, 10))

    self.priv_entry = tk.Entry(priv_panel, font=self.font_mono,
bg="#F9FAFB", fg=self.text_main, relief="flat")
    self.priv_entry.pack(fill=tk.X, ipady=5)

# --- WORKSPACE AREA ---
workspace = tk.Frame(main_pad, bg=self.card_color, padx=20,
pady=20, highlightbackground="#E5E7EB", highlightthickness=1)
    workspace.pack(fill=tk.BOTH, expand=True)

# Input Section
    tk.Label(workspace, text="INPUT DATA", font=self.font_title,
bg=self.card_color, fg=self.text_main).pack(anchor=tk.W)
    tk.Label(workspace, text="Enter plaintext string to encrypt, or
paste numeric ciphertext to decrypt.", font=("Segoe UI", 10),
bg=self.card_color, fg=self.text_muted).pack(anchor=tk.W, pady=(0, 10))
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
self.txt_input = tk.Text(workspace, font=self.font_text, height=6,
bg="#F9FAFB", fg=self.text_main, relief="flat",
highlightbackground="#D1D5DB", highlightthickness=1)
self.txt_input.pack(fill=tk.X, pady=(0, 20))

# Action Buttons
btn_frame = tk.Frame(workspace, bg=self.card_color)
btn_frame.pack(fill=tk.X, pady=(0, 20))

btn_encrypt = tk.Button(btn_frame, text="🔒 ENCRYPT",
font=self.font_title, bg=self.accent_blue, fg="white",
relief="flat", padx=20, pady=10,
cursor="hand2", command=self.do_encrypt)
btn_encrypt.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=(0,
10))

btn_decrypt = tk.Button(btn_frame, text="🔓 DECRYPT",
font=self.font_title, bg=self.accent_green, fg="white",
relief="flat", padx=20, pady=10,
cursor="hand2", command=self.do_decrypt)
btn_decrypt.pack(side=tk.RIGHT, fill=tk.X, expand=True, padx=(10,
0))

# Output Section
output_header = tk.Frame(workspace, bg=self.card_color)
output_header.pack(fill=tk.X, pady=(0, 10))
tk.Label(output_header, text="OUTPUT TERMINAL",
font=self.font_title, bg=self.card_color,
fg=self.text_main).pack(side=tk.LEFT)

btn_copy = tk.Button(output_header, text="📄 Copy Output",
font=("Segoe UI", 9, "bold"), bg="#E5E7EB", fg=self.text_main,
relief="flat", padx=10, cursor="hand2",
command=self.copy_output)
btn_copy.pack(side=tk.RIGHT)

self.txt_output = tk.Text(workspace, font=self.font_mono,
height=7, bg="#111827", fg="#10B981", relief="flat", padx=10, pady=10,
state=tk.DISABLED)
```

[illegible]

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
def do_encrypt(self):
    raw_input = self.txt_input.get("1.0", tk.END).strip()
    if not raw_input:
        messagebox.showwarning("Input Required", "Please enter text
into the Input Data field.")
        return

    cipher_array = encrypt(self.public_key, raw_input)
    self.update_output(str(cipher_array))

def do_decrypt(self):
    raw_input = self.txt_input.get("1.0", tk.END).strip()
    if not raw_input:
        messagebox.showwarning("Input Required", "Please paste
encrypted numbers into the Input Data field.")
        return

    # BULLETPROOF PARSING: Use Regex to find all numbers, completely
    ignoring brackets, commas, or letters
    number_strings = re.findall(r'\d+', raw_input)

    if not number_strings:
        messagebox.showerror("Decryption Failed", "No numbers found in
the input. Ensure you pasted the ciphertext correctly.")
        return

    try:
        ciphertext = [int(num) for num in number_strings]
        decrypted_text = decrypt(self.private_key, ciphertext)
        self.update_output(decrypted_text)
    except Exception as e:
        messagebox.showerror("Decryption Error", f"An error occurred
while decrypting:\n{str(e)}")

# =====
# 4. APPLICATION LAUNCHER
# =====
if __name__ == "__main__":
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
root = tk.Tk()
app = EnterpriseRSAUI(root)
root.mainloop()
```

Output

